

Задание 3.1. Создание приложения с распараллеливанием данных. Parallel.For

Аннотация: В рамках данного практического задания будет рассмотрен пример использования параллельных циклов Parallel.For и Parallel.Foreach для распараллеливания данных.

Parallel.For

1. Создадим консольное приложение и назовем его, к примеру, "ParallelForApplication":

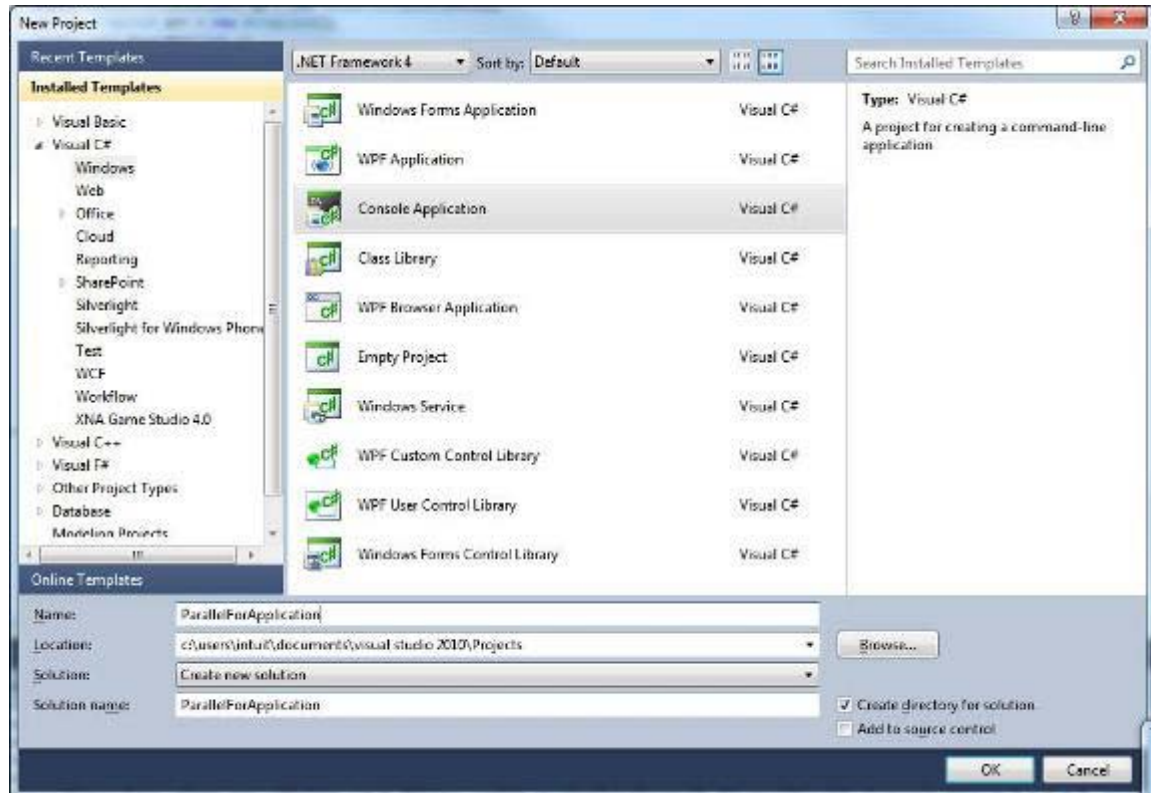


Рис. 1.

2. Создадим метод MaxNumber(), который бы находил максимальное случайно-сгенерированное число:

```
static int Max=1;
static void Maxnumber(int c)
{
    Random rnd = new Random();
    int v = rnd.Next(0, c);

    if (Max < v)
    {
        Max = v;
    }
}
```

3. В методе Main() создадим цикл, который будет передавать значения переменной i в метод Maxnumber:

```
for (int i = 1; i <= 10000000; i++)
{
    Maxnumber(i);
}
Console.WriteLine("Максимальное число: {0} ",Max);
Console.ReadLine();
```

4. Запустим программу. В результате, через некоторое время, отобразиться результат:

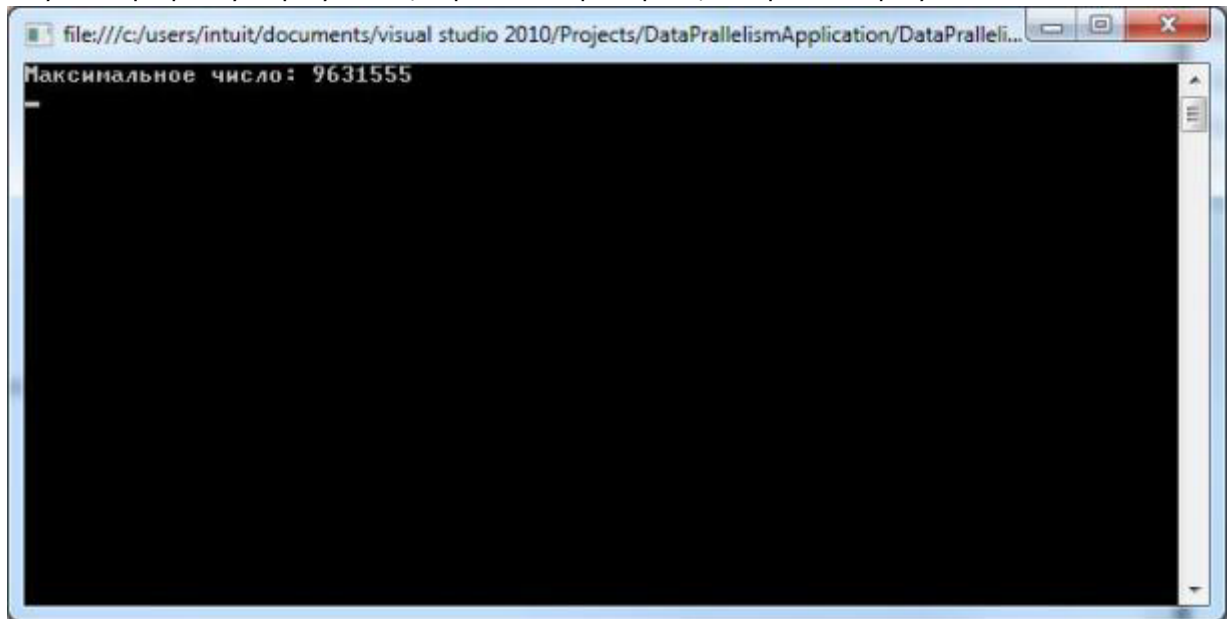


Рис. 10.2.

Листинг кода программы:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
namespace ParallelForApplication
{
    class Program
    {
        static int Max=1;
        static void Maxnumber(int c)
        {
            Random rnd = new Random();
            int v = rnd.Next(0, c);

            if (Max < v)
            {
                Max = v;
            }
        }
        static void Main(string[] args)
        {
            for (int i = 1; i <= 100000000; i++)
            {
                Maxnumber(i);
            }
            Console.WriteLine("Максимальное число: {0} ",Max);
            Console.ReadLine();
        }
    }
}
```

5. Теперь для того, что бы посмотреть, сколько по времени отработывается метод `Maxnumber()`, добавим в код строчки:

```
System.Diagnostics.Stopwatch sw =  
System.Diagnostics.Stopwatch.StartNew();  
    for (int i = 1; i <= 100000000; i++)  
    {  
        Maxnumber(i);  
    }  
    long elapsed = sw.ElapsedMilliseconds;  
    Console.WriteLine("Время выполнения алгоритма в миллисекундах:  
{0}", elapsed);  
    Console.WriteLine("Максимальное число: {0} ", Max);  
    Console.ReadLine();
```

6. Запустим программу. В результате отобразиться следующее:

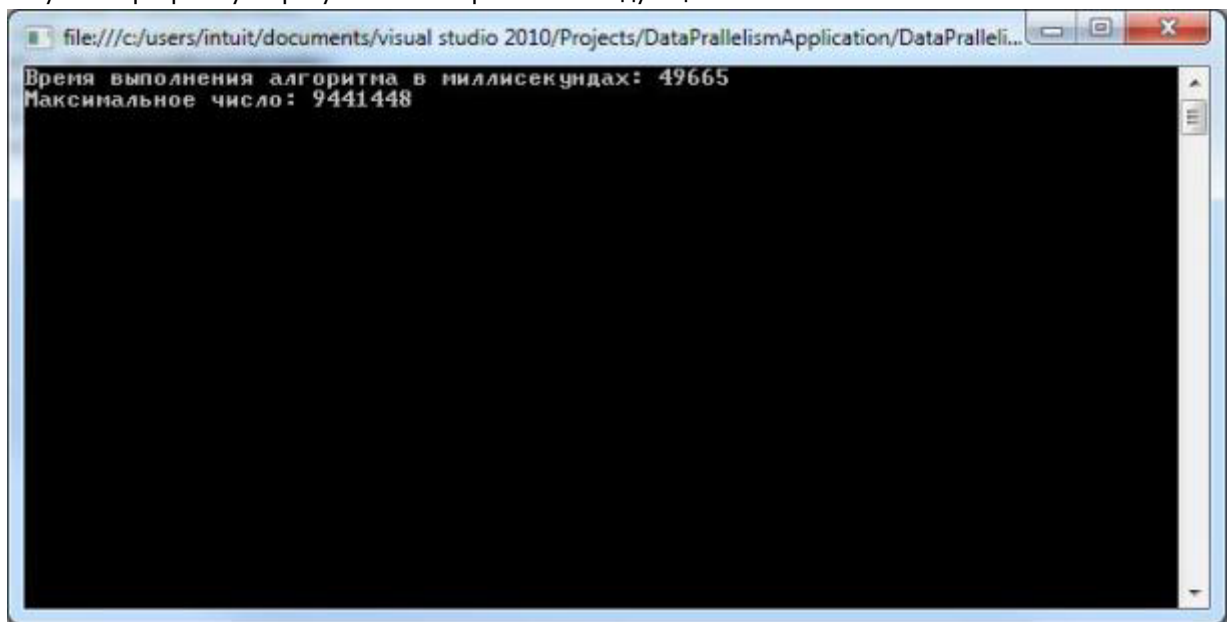


Рис. 3.

Примечание. Как видно из результата, время выполнения алгоритма составляет ~50 секунд.

7. Подключим директиву, для того что бы использовать цикл `Parallel.For`:
- ```
using System.Threading.Tasks
```
8. Теперь заменим цикл `for` на `Parallel.For`, для того, что бы метод `Maxnumber`, выполнялся не последовательно, а асинхронно:
- ```
Parallel.For(1, 100000000, i =>  
    {  
        Maxnumber(i);  
    });
```
9. Запустим программу. В результате отобразится следующее:

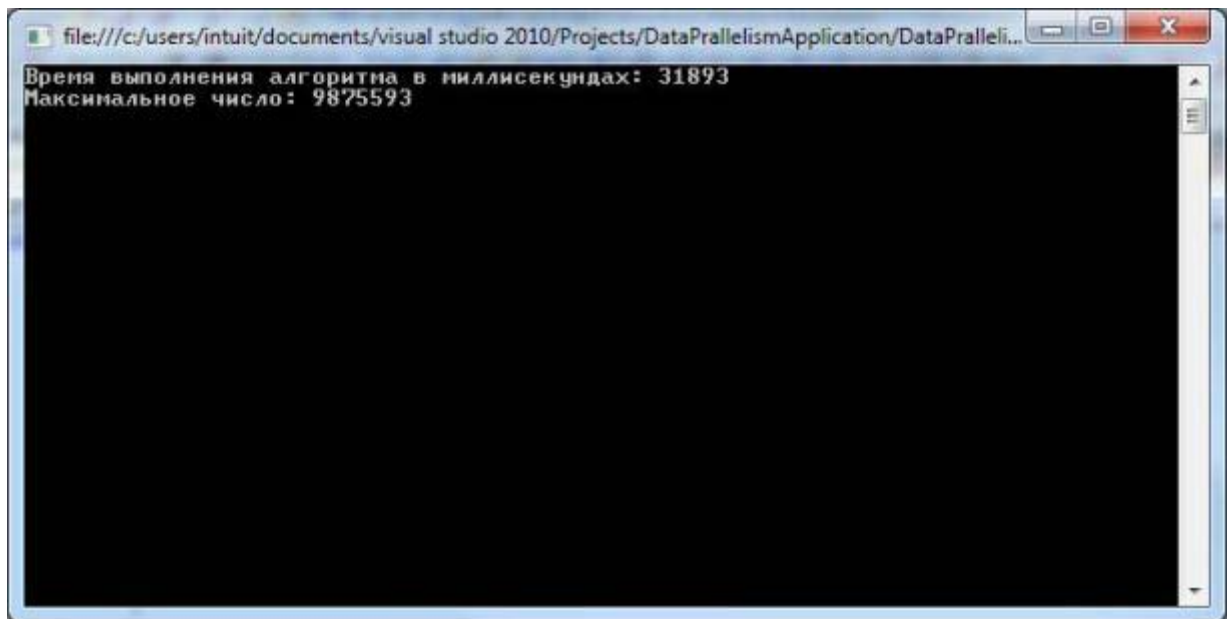


Рис. 4.

Примечание. Как видно из результата выполнения программы, время выполнения распараллеленного метода `Maxnumber()`, составит приблизительно 31 секунду.

Листинг кода программы:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace ParallelForApplication
{
    class Program
    {
        static int Max=1;
        static void Maxnumber(int c)
        {
            Random rnd = new Random();
            int v = rnd.Next(0, c);

            if (Max < v)
            {
                Max = v;
            }
        }
        static void Main(string[] args)
        {
            System.Diagnostics.Stopwatch sw =
            System.Diagnostics.Stopwatch.StartNew();
            Parallel.For(1, 10000000, i =>
            {
                Maxnumber(i);
            });
        }
    }
}
```

```

    });
    long elapsed = sw.ElapsedMilliseconds;
    Console.WriteLine("Время выполнения алгоритма в
миллисекундах: {0}", elapsed);
    Console.WriteLine("Максимальное число: {0} ", Max);
    Console.ReadLine();
}
}
}

```

Задание 3.2. Создание приложения с распараллеливанием данных. Parallel.Foreach

Аннотация: В рамках данного практического задания будет рассмотрен пример использования параллельных циклов Parallel.For и Parallel.Foreach для распараллеливания данных.

Parallel.Foreach

В данной части практического занятия будет разработано консольное приложение, которое скачивает содержание Web-страниц, в цикле for each, и в цикле Parallel.Foreach.

1. Создадим консольное приложение и назовем его, к примеру, ParallelForeachApplication

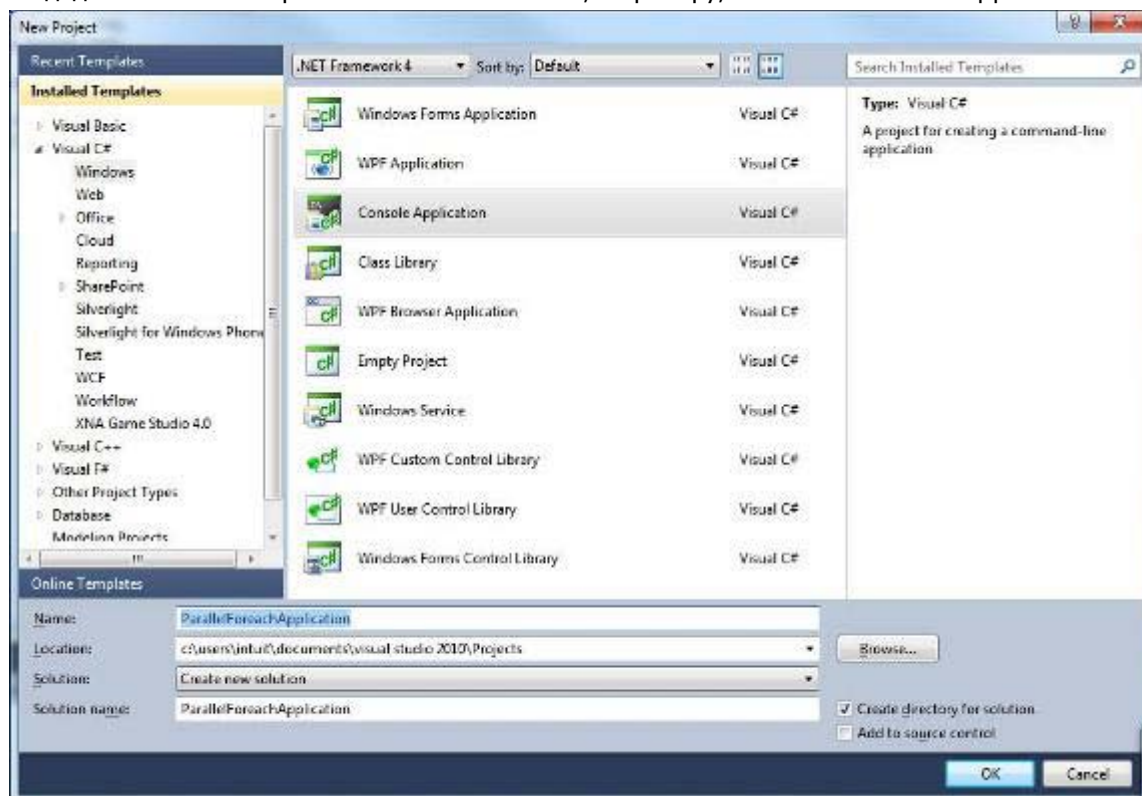


Рис. 5.

2. Подключаем необходимые директивы:
`using System.Net;`
`using System.Threading.Tasks;`
3. Инициализируем коллекцию ссылок List<string>:
`List<string> urls = new List<string>()`
`{ "http://intuit.ru",`
 `"http://rbc.ru",`
 `"http://ozon.ru",`
 `"http://google.com",`
 `"http://mail.ru",`
 `"http://lenta.ru"`
`};`
4. Теперь, создадим цикл for each, в котором бы скачивалась содержание Web-страниц, определенные в списке List<string>:
`System.Diagnostics.Stopwatch sw =`
`System.Diagnostics.Stopwatch.StartNew();`

```

foreach (string url in urls)
{
    WebClient client = new WebClient();
    Console.WriteLine("Скачиваем : {0}" , url);
    client.DownloadString(url);
}

long elapsed = sw.ElapsedMilliseconds;
Console.WriteLine("Затраченное время в миллисекундах: {0}", elapsed);
sw.Stop();

```

5. Запустим программу. На экране отобразится результат выполнения алгоритма:

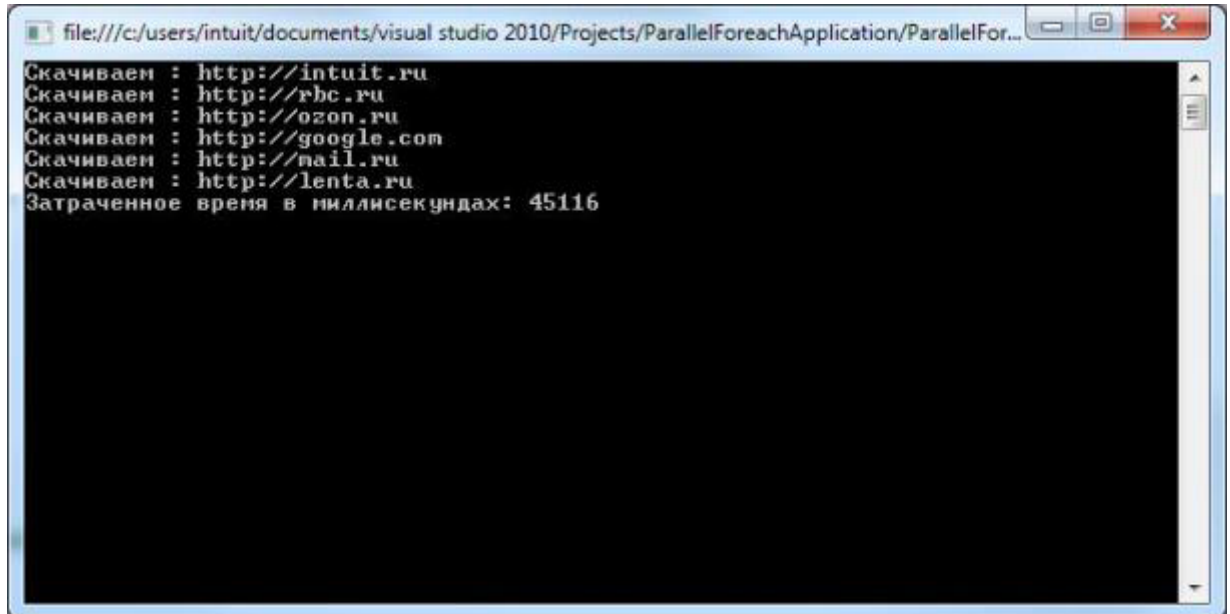


Рис. 6.

6. Далее, создадим аналогичный цикл, который бы скачивал содержание Web-страниц, но в параллельном режиме:

```

Console.WriteLine("=====");
sw.Restart();
Parallel.ForEach(urls, url =>
{
    WebClient client = new WebClient();
    Console.WriteLine("Скачиваем : " + url);
    client.DownloadString(url);
});
elapsed = sw.ElapsedMilliseconds;
Console.WriteLine("Затраченное время в миллисекундах: {0}",
elapsed);
sw.Stop();

Console.ReadLine();

```

7. Запустим программу. В результате на экране отобразится следующее:

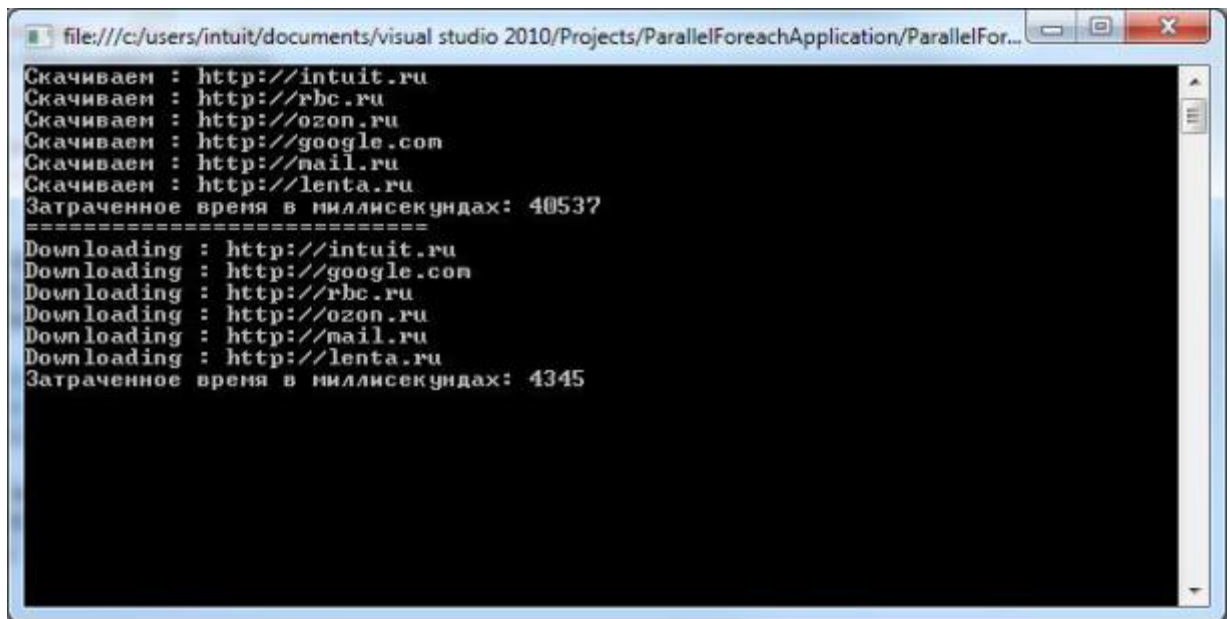


Рис. 7.

Примечание. Цикл `Parallel.Foreach`, будет выполняться значительно быстрее т.к. методы, прописанные внутри тела цикла, будут выполняться в несколько потоков.

Листинг кода программы:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Collections;
using System.Net;
using System.Threading.Tasks;
namespace ParallelForeachApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            List<string> urls = new List<string>() {
                "http://intuit.ru",
                "http://rbc.ru",
                "http://ozon.ru",
                "http://google.com",
                "http://mail.ru",
                "http://lenta.ru"
            };
            System.Diagnostics.Stopwatch sw =
                System.Diagnostics.Stopwatch.StartNew();

            foreach (string url in urls)
            {
                WebClient client = new WebClient();
                Console.WriteLine("Скачиваем : {0}" , url);
                client.DownloadString(url);
            }
        }
    }
}
```



```

long elapsed = sw.ElapsedMilliseconds;
Console.WriteLine("Затраченное время в миллисекундах: {0}",elapsed);
    sw.Stop();
    Console.WriteLine("=====");
    sw.Restart();
    Parallel.ForEach(urls, url =>
    {
        WebClient client = new WebClient();
        Console.WriteLine("Скачиваем : " + url);
        client.DownloadString(url);
    });
    elapsed = sw.ElapsedMilliseconds;
    Console.WriteLine("Затраченное время в миллисекундах:
{0}", elapsed);
        sw.Stop();
Console.ReadLine();
}
    }
}

```